



Assignment No. 2

Title:

NoSQL Database Implementation

Aim:

To explore and implement a NoSQL database system by installing and configuring MongoDB or Apache Cassandra, designing an appropriate schema for a specific use case, and performing CRUD operations to efficiently manage and manipulate data in a scalable database environment.

Objective:

The main objective of this assignment is to understand the concepts and practical implementation of NoSQL databases in handling large-scale and unstructured data. Students will explore and install a NoSQL database such as MongoDB or Apache Cassandra, design an appropriate schema for a specific use case, and perform basic CRUD (Create, Read, Update, Delete) operations. This assignment will help students understand the flexibility, scalability, and performance advantages of NoSQL databases compared to traditional relational database systems.

Software Requirements:

1. Operating System: Windows / Linux / macOS
2. MongoDB or Apache Cassandra
3. MongoDB Compass / Cassandra Query Language (CQL) Shell
4. Python 3.x (optional, for database connectivity)
5. Required libraries (e.g., pymongo for MongoDB or cassandra-driver for Cassandra)
6. Command Line / Terminal or IDE (VS Code / PyCharm)

Outcome:

After completing this lab, students will be able to:

1. Understand the fundamentals of NoSQL databases and their advantages over traditional relational databases.
2. Install and configure MongoDB or Apache Cassandra.
3. Design a suitable schema for a real-world use case in a NoSQL environment.
4. Perform CRUD (Create, Read, Update, Delete) operations and analyze how data is stored and managed in NoSQL databases.

Theory:

Modern applications generate large volumes of structured, semi-structured, and unstructured data that cannot always be efficiently managed using traditional relational databases. NoSQL databases are designed to handle such data by providing flexible schemas, horizontal scalability, and high performance in distributed environments. They are widely used in big data systems, real-time



applications, and cloud-based platforms. NoSQL databases are especially useful when dealing with rapidly changing data structures and massive amounts of data that require quick access and processing.

1. **Flexible Data Model**

Unlike relational databases that use fixed table structures, NoSQL databases allow dynamic schemas. For example, MongoDB stores data in JSON-like documents, where each record can have a different structure. This flexibility makes it easier to modify and scale applications without redesigning the entire database schema. Developers can easily add or remove attributes in a document without affecting other records. This adaptability is particularly useful for applications that evolve frequently and require rapid development cycles.

2. **Distributed Architecture and Scalability**

NoSQL databases are designed to scale horizontally by distributing data across multiple servers. Apache Cassandra is built for high availability and fault tolerance, ensuring that data remains accessible even if some nodes fail. This makes NoSQL systems suitable for handling large-scale and high-traffic applications. Horizontal scaling allows organizations to expand their systems by simply adding more servers, which improves performance and storage capacity without major infrastructure changes.

3. **Schema Design for Use Case**

In NoSQL systems, schema design is based on application requirements rather than strict normalization rules. Data is structured according to how it will be queried and accessed, improving performance and efficiency. This design approach reduces the need for complex joins and multiple table relationships, which are common in relational databases.

4. **CRUD Operations**

NoSQL databases support basic operations such as **Create, Read, Update, and Delete**.

- These operations allow efficient data insertion, retrieval, modification, and removal, helping users understand how data is managed in a scalable database environment. For example, developers can insert new documents into a collection, retrieve specific records using queries, update existing data, or remove outdated information. These operations form the foundation for most database-driven applications.

6. **Types of NoSQL Databases**

NoSQL databases are categorized into several types depending on how they organize and store data:



- **Document-based databases** store data in flexible document formats such as JSON or BSON. MongoDB is a common example.
- **Key-value stores** store data as simple key-value pairs and provide extremely fast data access. Redis and DynamoDB are examples.
- **Column-family stores** organize data in columns instead of rows, making them efficient for large-scale analytical processing. Cassandra and HBase fall into this category.
- **Graph databases** focus on relationships between data elements and are widely used in social networks, recommendation engines, and fraud detection systems. Neo4j is an example.

7. **High Availability and Fault Tolerance**

NoSQL systems ensure high availability through replication and distributed storage mechanisms. Data is automatically copied across multiple nodes in the system so that if one node fails, the data can still be accessed from another node. This capability makes NoSQL databases reliable for mission-critical applications that require continuous availability.

8. **Performance Optimization**

NoSQL databases are optimized for high-speed data processing and retrieval. Techniques such as indexing, caching, and data partitioning are used to improve performance. These systems are designed to handle thousands or even millions of operations per second, making them suitable for applications with heavy user traffic.

9. **Integration with Big Data Technologies**

NoSQL databases often integrate with big data tools and frameworks such as Apache Hadoop and Apache Spark. This integration allows organizations to perform advanced data analytics, machine learning, and real-time data processing. For example, data stored in MongoDB or Cassandra can be analyzed using Spark for large-scale data processing tasks.

10. **Real-World Applications**

NoSQL databases are widely used in many industries and applications. Social media platforms use them to manage user profiles and interactions. E-commerce companies use them to handle product catalogs and customer transactions. IoT systems rely on NoSQL databases to store and process data from millions of connected devices. Streaming services and gaming platforms also use NoSQL technologies to manage large volumes of real-time data.

11. **Advantages over Traditional Databases**

Compared to traditional relational databases, NoSQL databases provide better scalability,



flexibility, and performance for modern applications. They reduce the complexity of database design, support distributed architectures, and allow developers to build applications that can handle massive datasets efficiently.

Overall, NoSQL databases play a vital role in modern data management systems. Their ability to support flexible data models, distributed processing, and high scalability makes them an essential component of big data ecosystems and modern cloud-based applications.

To explore and implement a NoSQL database system by installing and configuring MongoDB or Apache Cassandra, designing an appropriate schema for a specific use case, and performing CRUD operations to efficiently manage and manipulate data in a scalable database environment.

Algorithm / Steps:

1. Install and configure MongoDB or Apache Cassandra on the local system.
2. Start the database server and open the MongoDB shell / CQL shell to establish a connection.
3. Create a new database for the selected use case (e.g., Student Management / E-commerce System).
4. Design and create a collection (MongoDB) or table (Cassandra) with appropriate fields.
5. based on the use case requirements.
6. Perform **Create** operation by inserting sample records into the collection/table.
7. Perform **Read** operation to retrieve and display stored records.
8. Perform **Update** operation to modify specific records based on given conditions.
9. Perform Delete operation to remove selected records from the database.
10. Verify the changes after each operation to ensure proper data management.

Source Code:

```
// 1. Create / Switch Database
use studentDB

// 2. Create Collection & Insert (Create)
// Insert single document
db.students.insertOne({
  roll_no: 101,
  name: "Amit",
  course: "BCA",
  marks: 85,
  semester: 3,
  subjects: ["Math", "Computer", "English"]
})
```



```
}}
```

```
// Insert multiple documents
```

```
db.students.insertMany([  
  {  
    roll_no: 102,  
    name: "Priya",  
    course: "MCA",  
    marks: 92,  
    semester: 5,  
    subjects: ["DBMS", "Java", "Networking"]  
  },  
  {  
    roll_no: 103,  
    name: "Rahul",  
  
    course: "BCA",  
    marks: 78,  
    semester: 3,  
    subjects: ["Math", "C++", "English"]  
  },  
  {  
    roll_no: 104,  
    name: "Neha",  
    course: "MCA",  
    marks: 88,  
    semester: 5,  
    subjects: ["Python", "DBMS", "OS"]  
  }  
])
```

```
// 3. Read Data (Read)
```

```
// Find all documents
```

```
db.students.find()
```

```
// Find with condition
```



```
db.students.find({ course: "BCA" })  
// Find with pretty formatting  
db.students.find().pretty()  
// Find specific fields  
db.students.find({}, { name: 1, marks: 1, _id: 0 })  
// Find with comparison operators  
db.students.find({ marks: { $gt: 80 } })  
// 4. Update Record (Update)  
// Update single document  
db.students.updateOne(  
    { roll_no: 101 },  
    { $set: { marks: 90 } }  
)  
// Update multiple documents  
db.students.updateMany(  
    { course: "BCA" },  
    { $set: { department: "Computer Science" } }  
)  
// Increment value  
db.students.updateOne(  
    { roll_no: 102 },  
    { $inc: { marks: 5 } }  
)  
// Add field to existing document  
db.students.updateOne(  
    { roll_no: 103 },  
    { $set: { attendance: "85%" } }  
)  
// 5. Delete Record (Delete)  
// Delete single document  
db.students.deleteOne({ roll_no: 101 })  
// Delete multiple documents  
db.students.deleteMany({ course: "BCA" })  
// Delete all documents (but keep collection)
```



```
db.students.deleteMany({})
```

```
// 6. Additional Operations
```

```
// Count documents
```

```
db.students.countDocuments()
```

```
// Sort results
```

```
db.students.find().sort({ marks: -1 })
```

```
// Limit results
```

```
db.students.find().limit(2)
```

```
// Drop collection
```

```
db.students.drop()
```

```
// Show all collections
```

```
show collections
```

Conclusion:

This experiment demonstrates the practical implementation of NoSQL databases for managing flexible and large-scale data. By installing and working with MongoDB or Apache Cassandra, we understood how schema design differs from traditional relational databases. Performing CRUD operations provided hands-on experience in creating, retrieving, updating, and deleting records efficiently. This experiment highlights the importance of NoSQL databases in modern applications that require scalability, high availability, and dynamic data structures.

Questions

1. What is a NoSQL database and how does it differ from relational databases?
2. Explain the features and advantages of MongoDB.
3. What is the role of schema design in NoSQL databases?
4. Describe the CRUD operations with examples in a NoSQL database.
5. Why are NoSQL databases preferred for big data and distributed applications?